

Chapter 12 Fine-Tuning Generation Models

Instruction Tuning with QLoRA

```

# Import AutoTokenizer to load the model's tokenizer
from transformers import AutoTokenizer

# Import load_dataset to load a dataset from Hugging Face
from datasets import load_dataset

# Load the tokenizer and chat template from TinyLlama
template_tokenizer = AutoTokenizer.from_pretrained(
    "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
)

# Define a function to format each conversation
def format_prompt(example):

    # Get the conversation messages from the example
    chat = example["messages"]

    # Apply TinyLlama's chat template to the messages
    prompt = template_tokenizer.apply_chat_template(
        chat,
        tokenize=False
    )

    # Return the formatted prompt as text, not tokens
    return {"text": prompt}

# Load the UltraChat 200k dataset
dataset = (
    load_dataset(
        "HuggingFaceH4/ultrachat_200k",

        # Use the test_sft portion of the dataset
        split="test_sft"
    )

    # Shuffle the dataset using a fixed seed
    .shuffle(seed=42)

    # Select only the first 3000 examples
    .select(range(3000))
)

# Apply the format_prompt function to all examples
dataset = dataset.map(format_prompt)

# Print the formatted conversation at index 2576
print(dataset["text"][2576])

```

```

Map: 100%                               3000/3000 [00:02<00:00, 2507.22 examples/s]

<|user|>
Given the text: Knock, knock. Who's there? Hike.
Can you continue the joke based on the given text material "Knock, knock. Who's there? Hike"?</s>
<|assistant|>
Sure! Knock, knock. Who's there? Hike. Hike who? Hike up your pants, it's cold outside!</s>
<|user|>
Can you tell me another knock-knock joke based on the same text material "Knock, knock. Who's there? Hike"?</s>
<|assistant|>
Of course! Knock, knock. Who's there? Hike. Hike who? Hike your way over here and let's go for a walk!</s>

```

Model Quantization

```

# Install or upgrade bitsandbytes to version 0.46.1 or newer
!pip install -U bitsandbytes>=0.46.1

```

```

# Import PyTorch for tensor operations and model computation
import torch

# Import AutoModelForCausalLM to load a language model for text generation
# Import AutoTokenizer to convert text into tokens
# Import BitsAndBytesConfig to configure 4-bit/8-bit quantization

```

```

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# Specify the TinyLlama model to load
model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"

# Create the configuration for reducing the model's memory usage
bnb_config = BitsAndBytesConfig(

    # Load the model using 4-bit quantization
    load_in_4bit=True,

    # Use NF4 quantization, which is designed for neural network weights
    bnb_4bit_quant_type="nf4",

    # Use float16 for calculations during model execution
    bnb_4bit_compute_dtype="float16",

    # Apply an additional quantization step to save more memory
    bnb_4bit_use_double_quant=True,
)

# Load the language model
model = AutoModelForCausalLM.from_pretrained(

    model_name,

    # Automatically place the model on available devices such as the GPU
    device_map="auto",

    # Apply the 4-bit quantization configuration
    quantization_config=bnb_config,
)

# Disable the key-value cache during training to reduce memory usage
model.config.use_cache = False

# Set the tensor parallelism value to 1
model.config.pretraining_tp = 1

# Load the tokenizer associated with the model
tokenizer = AutoTokenizer.from_pretrained(
    model_name,

    # Allow custom model/tokenizer code if required by the model
    trust_remote_code=True
)

# Set <PAD> as the padding token
tokenizer.pad_token = "<PAD>"

# Add padding to the left side of the input
tokenizer.padding_side = "left"

```

LoRA Configuration

```

# Import LoRA configuration and PEFT preparation functions
from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model

# Prepare LoRA Configuration
peft_config = LoraConfig(

    # Controls how strongly the LoRA updates affect the original model
    lora_alpha=32,

    # Randomly drops 10% of LoRA values during training to reduce overfitting
    lora_dropout=0.1,

    # Sets the rank (size) of the LoRA matrices
    r=64,

    # Do not train additional bias parameters
    bias="none",

    # Specifies that the model is a causal language model
    task_type="CAUSAL_LM",

    # Specifies the model layers where LoRA adapters will be added
    target_modules=
    # Layers to target
    target_modules=

```

```

        "k_proj",      # Key projection layer
        "gate_proj",  # Gate projection layer
        "v_proj",     # Value projection layer
        "up_proj",    # Up projection layer
        "q_proj",     # Query projection layer
        "o_proj",     # Output projection layer
        "down_proj"   # Down projection layer
    ]
)

# Prepare the 4-bit quantized model for efficient training
model = prepare_model_for_kbit_training(model)

# Add the LoRA adapters to the model
model = get_peft_model(model, peft_config)

```

Training Configuration

```

from transformers import TrainingArguments

# Set the folder where training results will be saved
output_dir = "./results"

# Define the training settings
training_arguments = TrainingArguments(
    # Folder to save model checkpoints and results
    output_dir=output_dir,

    # Number of examples processed at once per GPU
    per_device_train_batch_size=2,

    # Accumulate gradients for 4 steps before updating the model
    gradient_accumulation_steps=4,

    # Use a memory-efficient AdamW optimizer
    optim="paged_adamw_32bit",

    # Set the learning rate
    learning_rate=2e-4,

    # Use cosine learning rate scheduling
    lr_scheduler_type="cosine",

    # Train for 1 complete epoch
    num_train_epochs=1,

    # Log training information every 10 steps
    logging_steps=10,

    # Use 16-bit precision to reduce memory usage
    fp16=True,

    # Reduce memory usage during training
    gradient_checkpointing=True
)

```

Training

```

# Remove the current TRL version and install TRL 0.12.2
!pip uninstall -y trl
!pip install trl==0.12.2

```



```

WARNING: Skipping trl as it is not installed.
Collecting trl==0.12.2
  Downloading trl-0.12.2-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: accelerate>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from trl==0.12.2) (1.14.0)
Requirement already satisfied: datasets>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from trl==0.12.2) (4.0.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from trl==0.12.2) (13.9.4)
Collecting transformers<4.47.0 (from trl==0.12.2)
  Downloading transformers-4.46.3-py3-none-any.whl.metadata (44 kB)
44.1/44.1 kB 4.1 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2) (5.9.8)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2) (6.0.2)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2)
Requirement already satisfied: huggingface_hub>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.34.0->trl==0.12.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets>=2.21.0->trl==0.12.2) (3.2.0)

# Import SFTTrainer for supervised fine-tuning
from trl import SFTTrainer

# Create the supervised fine-tuning trainer
trainer = SFTTrainer(
    # Use the prepared TinyLlama model
    model=model,

    # Use the formatted training dataset
    train_dataset=dataset,

    # Use the "text" column as training input
    dataset_text_field="text",

    # Use the model's tokenizer to process the text
    tokenizer=tokenizer,

    # Use the training settings defined earlier
    args=training_arguments,

    # Limit each input sequence to 512 tokens
    max_seq_length=512,

    # Use the LoRA configuration for parameter-efficient training
    peft_config=peft_config,
)

# Start training the model
trainer.train()

# Save the trained QLoRA adapter weights
trainer.model.save_pretrained("TinyLlama-1.1B-qlora")

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cut
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets>=2.2
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets>=2.21.0->trl=
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets>=2.21.0->trl=
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: cuda-pathfinder==1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dat
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0->accele
Downloading trl-0.12.2-py3-none-any.whl (365 kB)
365.7/365.7 kB 34.5 MB/s eta 0:00:00
Downloading transformers-4.46.3-py3-none-any.whl (10.0 MB)
10.0/10.0 MB 126.7 MB/s eta 0:00:00
Downloading huggingface_hub-0.36.2-py3-none-any.whl (566 kB)
566.4/566.4 kB 42.3 MB/s eta 0:00:00
Downloading tokenizers-0.20.3-cp312-cp312-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (3.0 MB)
3.0/3.0 MB 112.8 MB/s eta 0:00:00
Installing collected packages: huggingface_hub, tokenizers, transformers, trl
  Attempting uninstall: huggingface_hub
    Found existing installation: huggingface_hub 1.23.0
    Uninstalling huggingface_hub-1.23.0:
      Successfully uninstalled huggingface_hub-1.23.0
  Attempting uninstall: tokenizers

```

```
Found existing installation: tokenizers 0.22.2
Uninstalling tokenizers-0.22.2:
  Successfully uninstalled tokenizers-0.22.2
Attempting uninstall: transformers
Found existing installation: transformers 5.13.1
Uninstalling transformers-5.13.1:
  Successfully uninstalled transformers-5.13.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
gradio 6.20.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.36.2 which is incompatible.
Successfully installed huggingface_hub-0.36.2 tokenizers-0.20.3 transformers-4.46.3 trl-0.12.2
WARNING: The following packages were previously imported in this runtime:
[huggingface_hub,tokenizers,transformers]
You must restart the runtime in order to use newly installed versions.
```

[RESTART SESSION](#)

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_deprecation.py:100: FutureWarning: Deprecated argument(s) u
Deprecated positional argument(s) used in SFTTrainer, please use the SFTConfig to set these arguments instead.
  warnings.warn(message, FutureWarning)
/usr/local/lib/python3.12/dist-packages/trl/trainer/sft_trainer.py:300: UserWarning: You passed a `max_seq_length` argumen
warnings.warn(
/usr/local/lib/python3.12/dist-packages/trl/trainer/sft_trainer.py:328: UserWarning: You passed a `dataset_text_field` arg

```

```
# Merge Weights
```

```

# Install torchao, a PyTorch library for model optimization and quantization
!pip install -U "torchao==0.17.0"

```

```

Collecting torchao==0.17.0
  Downloading torchao-0.17.0-cp310-cp310-manylinux_2_24_x86_64.whl.metadata (20 kB)
  Downloading torchao-0.17.0-cp310-cp310-manylinux_2_24_x86_64.whl (3.2 MB)
  Installing collected packages: torchao
  Successfully installed torchao-0.17.0
  WARNING: Running pip as the 'root' user can result in broken permissions and conflicting usage with another
  installed package. It is recommended to create a virtual environment instead: https://pip.pypa.io/encl
  Tracking the installation for debugging purposes: /root/.cache/pip/ephemeral-report/torchao-0.17.0
  WARNING: The python packages were previously imported in this runtime.
  You must restart the runtime in order to use newly installed versions.

```

RESTART SESSION

Loss

```

# Import AutoPeftModelForCausalLM to load the saved PEFT/QLoRA model
from peft import AutoPeftModelForCausalLM

```

```

# Load the saved QLoRA model from the specified folder
model = AutoPeftModelForCausalLM.from_pretrained(

```

```

    # Specify the folder containing the saved QLoRA model
    "TinyLlama-1.1B-qlora",

```

```

    # Reduce CPU memory usage while loading the model
    low_cpu_mem_usage=True,

```

```

    # Automatically place the model on available devices
    device_map="auto",
)

```

```

# Merge the LoRA weights with the base model and remove the adapter
merged_model = model.merge_and_unload()

```

```

WARNING:torchao:Failed to load /usr/local/lib/python3.12/dist-packages/torchao/_C_mxfp8.cpython-310-x86_64-linux-gnu.so: Cou
WARNING:torchao:Failed to load /usr/local/lib/python3.12/dist-packages/torchao/_C_cutlass_90a.abi3.so: Could not load this l
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate to the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
180      1.325800

```

```

# Import pipeline to easily run text generation with the model
from transformers import pipeline

```

```

# Create a prompt for the instruction-tuned model
prompt = """<user|>
Tell me something about Large Language Models.</s>
<|assistant|>
"""

```

```

# Create a text-generation pipeline using our merged model
pipe = pipeline(

```

```

    # Specify that the task is text generation
    task="text-generation",

```

```

    # Use the merged fine-tuned model
    model=merged_model,

```

```

    # Use the tokenizer for converting text to tokens
    tokenizer=tokenizer)

```

```

# Generate a response and print the generated text
print(pipe(prompt)[0]["generated_text"])

```

340

1.383300

```

350      1.310700
360      1.441000
370      1.445300
380      1.445300
390      1.445300
400      1.445300
410      1.445300
420      1.445300
430      1.445300
440      1.445300
450      1.445300
460      1.445300
470      1.445300
480      1.445300
490      1.445300
500      1.445300
510      1.445300
520      1.445300
530      1.445300
540      1.445300
550      1.445300
560      1.445300
570      1.445300
580      1.445300
590      1.445300
600      1.445300
610      1.445300
620      1.445300
630      1.445300
640      1.445300
650      1.445300
660      1.445300
670      1.445300
680      1.445300
690      1.445300
700      1.445300
710      1.445300
720      1.445300
730      1.445300
740      1.445300
750      1.445300
760      1.445300
770      1.445300
780      1.445300
790      1.445300
800      1.445300
810      1.445300
820      1.445300
830      1.445300
840      1.445300
850      1.445300
860      1.445300
870      1.445300
880      1.445300
890      1.445300
900      1.445300
910      1.445300
920      1.445300
930      1.445300
940      1.445300
950      1.445300
960      1.445300
970      1.445300
980      1.445300
990      1.445300
1000     1.445300

```

Preference Tuning with DPO

```
# Templating Alignment Data
```

```
# Import load_dataset to load the DPO dataset from Hugging Face
from datasets import load_dataset

# Define a function to format each example for DPO training
def format_prompt(example):
    """Format the prompt using the <|user|> template TinyLlama is using"""

    # Format the prompt using TinyLlama's chat template
    system = "<|system|>\n" + example["system"] + "\n"

    # Create the user prompt and assistant section
    prompt = (
        "<|user|>\n"
        + example["input"]
        + "\n\n<|assistant|>\n"
    )

    # Add a newline to the preferred answer
    chosen = example["chosen"] + "\n"

    # Add a newline to the rejected answer
    rejected = example["rejected"] + "\n"

    # Return the formatted prompt, chosen answer, and rejected answer
    return {
        "prompt": system + prompt,
        "chosen": chosen,
        "rejected": rejected,
    }

# Apply formatting to the dataset and select relatively short answers
# Load the Intel Orca DPO preference dataset
dpo_dataset = load_dataset(
    "argilla/distilabel-intel-orca-dpo-pairs",
    split="train"
)

# Filter the dataset to keep high-quality and useful examples
dpo_dataset = dpo_dataset.filter(
    lambda r:

    # Remove examples where both answers are considered equal
    r["status"] != "tie"

    # Keep examples where the preferred answer has a score of at least 8
    and r["chosen_score"] >= 8

    # Remove examples that are already present in the GSM8K training set
    and not r["in_gsm8k_train"]
)

# Apply the formatting function to every example
dpo_dataset = dpo_dataset.map(

    # Convert each example into prompt, chosen, and rejected format
    format_prompt,

    # Remove the original columns after formatting
    remove_columns=dpo_dataset.column_names
)
```



```
# Display the formatted DPO dataset
dpo_dataset
```

```
README.md:      9.05k/? [00:00<00:00, 759kB/s]

data/train-00000-of-00001.parquet: 100%      79.2M/79.2M [00:03<00:00, 42.8MB/s]

Generating train split: 100%      12859/12859 [00:01<00:00, 11537.55 examples/s]

Filter: 100%      12859/12859 [00:00<00:00, 18115.04 examples/s]

Map: 100%      5922/5922 [00:01<00:00, 5099.52 examples/s]

Dataset({
  features: ['chosen', 'rejected', 'prompt'],
  num_rows: 5922
})
```

Model Quantization

```
# Import AutoPeftModelForCausalLM to load the saved QLoRA model
from peft import AutoPeftModelForCausalLM

# Import quantization configuration and tokenizer tools
from transformers import BitsAndBytesConfig, AutoTokenizer

# 4-bit quantization configuration - Q in QLoRA

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,          # Use 4-bit precision model loading
    bnb_4bit_quant_type="nf4",  # Quantization type
    bnb_4bit_compute_dtype="float16", # Compute dtype
    bnb_4bit_use_double_quant=True, # Apply nested quantization
)

# Load the saved QLoRA model
model = AutoPeftModelForCausalLM.from_pretrained(

    # Path to the saved QLoRA model
    "TinyLlama-1.1B-qlora",

    # Reduce CPU memory usage while loading
    low_cpu_mem_usage=True,

    # Automatically use available devices
    device_map="auto",

    # Apply the 4-bit quantization configuration
    quantization_config=bnb_config,
)

# Merge the LoRA weights with the base model
merged_model = model.merge_and_unload()

# Specify the original TinyLlama model name
model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"

# Load the tokenizer for the TinyLlama model
tokenizer = AutoTokenizer.from_pretrained(

    # Use the specified model
    model_name,

    # Allow custom tokenizer code if required
    trust_remote_code=True
)

# Use the EOS token as the padding token
tokenizer.pad_token = tokenizer.eos_token

# Add padding to the left side of the input
tokenizer.padding_side = "left"

/usr/local/lib/python3.12/dist-packages/peft/tuners/lora/bnb.py:373: UserWarning: Merge lora module to 4-bit linear may get
warnings.warn(
```

```
# Import LoRA configuration and functions for parameter-efficient training
from peft import (

    # Defines the LoRA settings
    LoraConfig,
```

```

        # Prepares the quantized model for training
        prepare_model_for_kbit_training,

        # Adds LoRA adapters to the model
        get_peft_model
    )

# Prepare LoRA configuration

peft_config = LoraConfig(

    # Set the LoRA scaling factor
    lora_alpha=32,

    # Set the dropout rate for LoRA layers
    lora_dropout=0.1,

    # Set the rank of the LoRA matrices
    r=64,

    # Do not train bias parameters
    bias="none",

    # Specify that the task is causal language modeling
    task_type="CAUSAL_LM",

    # Specify the model layers where LoRA adapters will be added
    target_modules=[
        "k_proj",
        "gate_proj",
        "v_proj",
        "up_proj",
        "q_proj",
        "o_proj",
        "down_proj"
    ]
)

# Prepare model for k-bit training
model = prepare_model_for_kbit_training(model)

# Add LoRA adapter to the model
model = get_peft_model(model, peft_config)

```

```

/usr/local/lib/python3.12/dist-packages/peft/tuners/tuners_utils.py:302: UserWarning: Already found a `peft_config` attribute
warnings.warn(

```

Training Configuration

```

from trl import DPOConfig

output_dir = "./results"

training_arguments = DPOConfig(
    output_dir=output_dir,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    optim="paged_adamw_32bit",
    learning_rate=1e-5,
    lr_scheduler_type="cosine",
    max_steps=200,
    logging_steps=10,
    fp16=True,
    gradient_checkpointing=True,
    warmup_ratio=0.1
)

```

Training

```

# Import DPOTrainer for Direct Preference Optimization training
from trl import DPOTrainer

# Create the DPO trainer
dpo_trainer = DPOTrainer(

    # Use the model for DPO fine-tuning
    model,

```

```
# Use the DPO training settings
args=training_arguments,

# Use the formatted preference dataset
train_dataset=dpo_dataset,

# Use the tokenizer to process the text
tokenizer=tokenizer,

# LoRA configuration can be enabled if required
# peft_config=peft_config,

# Control how strongly preference differences affect training
beta=0.1,

# Set the maximum length of the prompt
max_prompt_length=512,

# Set the maximum total sequence length
max_length=512,
)

# Start DPO fine-tuning
dpo_trainer.train()

# Save the trained DPO adapter weights
dpo_trainer.model.save_pretrained(
    "TinyLlama-1.1B-dpo-qlora"
)
```